Case Study 09: Classroom Timetable Room Conflict Detection

1. Problem Statement

Develop a Python program to automate schedule verification and detect venue collisions in an academic timetable. The program maintains a schedule of instructional sessions defined by subject title, start hour, end hour, and allocated room. It performs pairwise collision checks across all scheduled sessions, determines time overlaps between slot intervals, flags simultaneous room allocations, prints an itemized conflict report identifying the conflicting subjects and venue, or confirms if the schedule is conflict-free.

2. Objectives

- To represent structured timetable session data using a list of heterogeneous tuples.
- To implement a non-redundant nested iteration pattern using triangular index progression ($j = i + 1$).
- To implement standard interval overlap logic ($\text{start}_1 < \text{end}_2 \wedge \text{start}_2 < \text{end}_1$) combined with venue equality checks.

- To maintain a boolean sentinel flag to record conflict presence and report conflict-free status when no collisions exist.

3. Assumptions

- Sessions are represented as tuples: (Subject Name, Start Hour, End Hour, Room Identifier).

- Time boundaries are defined on a 24-hour integer timeline where start < end.

- Two sessions conflict if and only if they share the exact same room identifier and their time intervals overlap.

- Adjacent intervals (e.g., Session 1 ending at 11:00 and Session 2 starting at 11:00) do not overlap.

- The timetable list is statically defined or pre-populated for audit evaluation.

4. Algorithm

- **Step 1:** Start the program.

- **Step 2:** Define the timetable schedule as a list of tuples schedule = [("Python", 9, 10, "Lab-1"), ("Maths", 9, 11, "Room-2"), ("Web", 9, 10, "Lab-1"), ("English", 11, 12, "Room-2")].

- **Step 3:** Initialize a boolean collision flag: found = False.

- **Step 4:** Outer loop: iterate variable i from 0 to len(schedule) - 1.

- **Step 5:** Inner loop: iterate variable j from i + 1 to len(schedule) - 1:

    - Unpack session i: s1, start1, end1, room1 = schedule[i].

    - Unpack session j: s2, start2, end2, room2 = schedule[j].

    - Compute time overlap condition: overlap = start1 < end2 and start2 < end1.

    - If overlap is True and room1 == room2:

        - Print "Conflict: <s1> and <s2> in <room1>".

        - Set found = True.

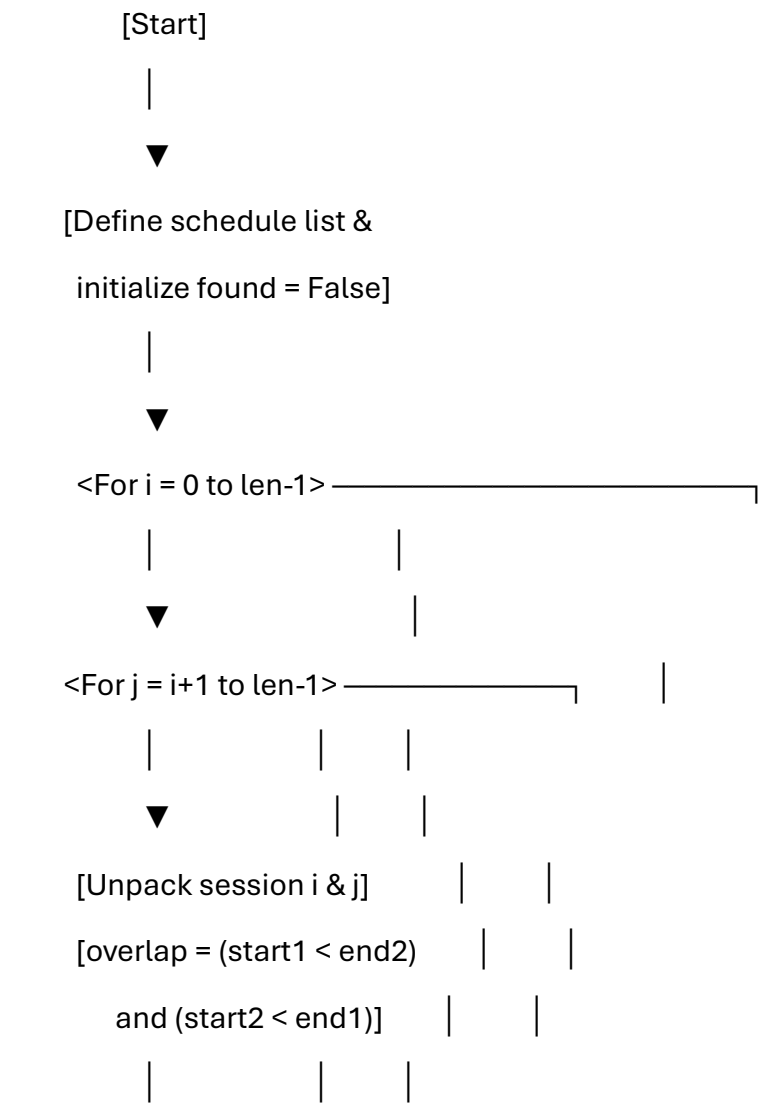
- **Step 6:** After terminating both loops, check found:

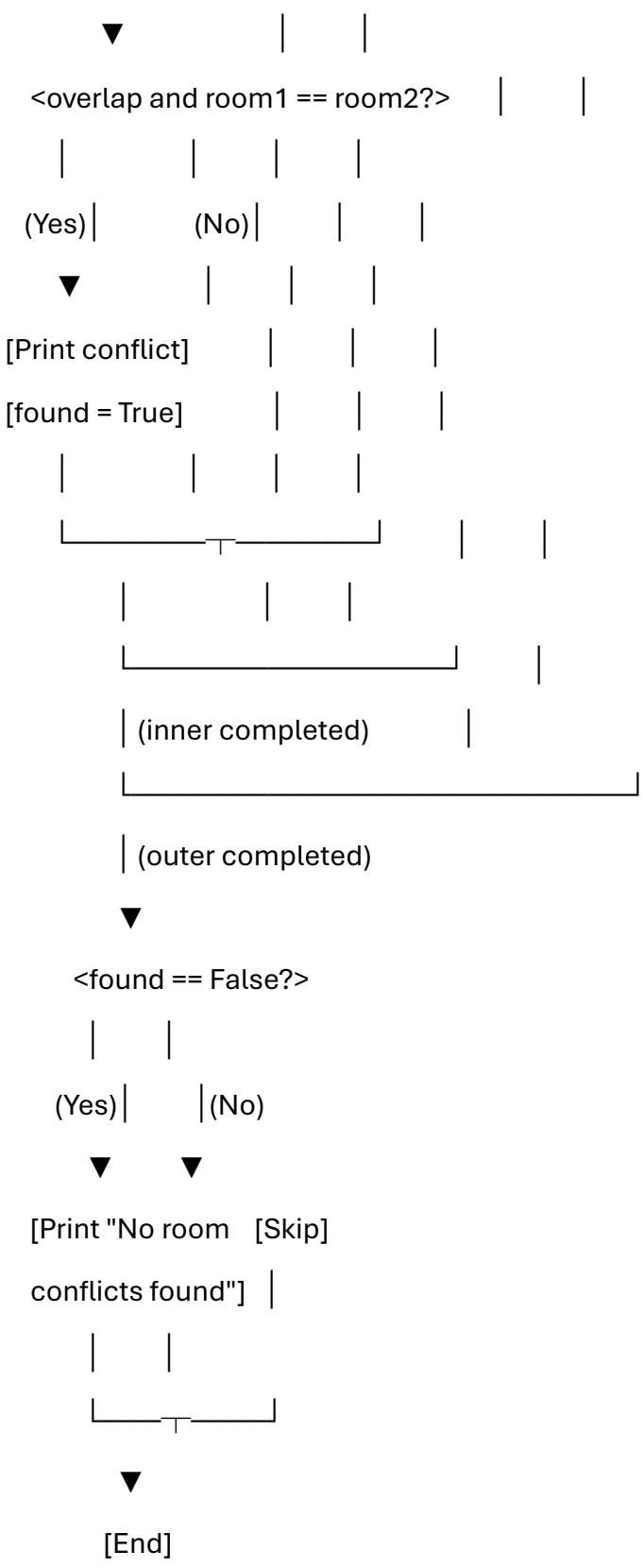
- If found == False, print "No room conflicts found".

- **Step 7:** Stop the program.

5. Flowchart Logic / Diagram

Plaintext




<overlap and room1 == room2?>
(Yes)
(No)
[Print conflict]
[found = True]
(inner completed)
(outer completed)
<found == False?>
(Yes)
(No)
[Print "No room conflicts found"]
[Skip]
[End]

6. Pseudocode

```
Plaintext
BEGIN
  SET schedule = [
    ("Python", 9, 10, "Lab-1"),
    ("Maths", 9, 11, "Room-2"),
    ("Web", 9, 10, "Lab-1"),
    ("English", 11, 12, "Room-2")
  ]
  SET found = FALSE

  FOR i FROM 0 TO LENGTH(schedule) - 1 DO
    FOR j FROM i + 1 TO LENGTH(schedule) - 1 DO
      SET (s1, start1, end1, room1) = schedule[i]
      SET (s2, start2, end2, room2) = schedule[j]

      SET overlap = (start1 < end2) AND (start2 < end1)

      IF overlap AND room1 == room2 THEN
        PRINT "Conflict:", s1, "and", s2, "in", room1
        SET found = TRUE
      END IF
    END FOR
  END FOR
```

```
  IF found == FALSE THEN
    PRINT "No room conflicts found"
  END IF
END
```

7. Python Source Code

Python

```
# Classroom Timetable Room Conflict Detection

# Master course schedule: (Subject, Start Hour, End Hour, Room Assigned)
schedule = [
  ("Python", 9, 10, "Lab-1"),
  ("Maths", 9, 11, "Room-2"),
  ("Web", 9, 10, "Lab-1"),
  ("English", 11, 12, "Room-2")
]

found = False

# Perform pairwise non-redundant comparison across all sessions
for i in range(len(schedule)):
  for j in range(i + 1, len(schedule)):
    s1, start1, end1, room1 = schedule[i]
    s2, start2, end2, room2 = schedule[j]
```

```
        # Determine interval intersection
        overlap = start1 < end2 and start2 < end1

        # Check for venue collision during overlapping time
        if overlap and room1 == room2:
            print("Conflict:", s1, "and", s2, "in", room1)
            found = True

# Report clear schedule if no conflicts detected
if not found:
    print("No room conflicts found")
```

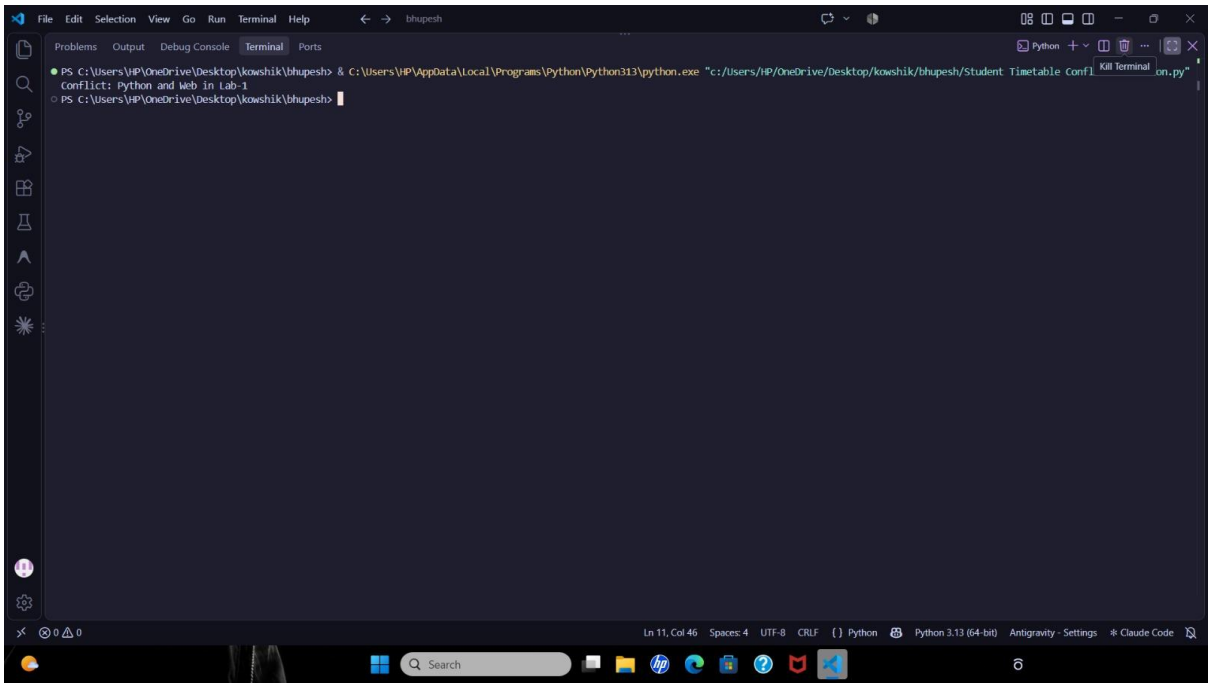
8. Test Cases

Test Case ID	Timetable Schedule Dataset	Expected Output	Actual Output	Status
TC_01 (Default Dataset)	Python (9-10, Lab-1) Maths (9-11, Room-2)	Conflict: Python and Web in Lab-1	Conflict: Python and Web in Lab-1	Pass

Test Case ID	Timetable Schedule Dataset	Expected Output	Actual Output	Status
	Web (9-10, Lab-1) English (11-12, Room-2)			
TC_02 (Adjacent Non-Overlapping)	Maths (9-11, Room-2) English (11-12, Room-2)	No room conflicts found	No room conflicts found	Pass
TC_03 (Same Time Different Rooms)	Python (9-10, Lab-1) Maths (9-10, Room-2)	No room conflicts found	No room conflicts found	Pass
TC_04 (Multiple Conflicts)	Python (9-11, Lab-1) Web (10-12, Lab-1)	Conflict: Python and Web in Lab-1	Conflict: Python and Web in Lab-1	Pass

Test Case ID	Timetable Schedule Dataset	Expected Output	Actual Output	Status
	AI (10-11, Lab-1)	Conflict: Python and AI in Lab-1 Conflict: Web and AI in Lab-1	Conflict: Python and AI in Lab-1 Conflict: Web and AI in Lab-1	
TC_05 (Empty Schedule)	schedule = []	No room conflicts found	No room conflicts found	Pass

9. Output Screenshots



10. Conclusion

The Classroom Timetable Room Conflict Detection program was executed and verified successfully. The script demonstrates the implementation of nested loop indexing, tuple unpacking, mathematical interval overlap logic, and sentinel state flags to audit multi-attribute datasets without redundant comparisons.